AIOracleManager.sol

AIOracleManager.sol

Purpose:

Register verified AI oracles or evaluators

Gate access to post-scoring + verification roles

Track stats or behavior for auditability

Used in SoGoodFeed, ProposalManager, or yield rebalancing

Key Features:

Add/remove oracle agents

Enable/disable

Optional metadata (e.g. model version, score method)

Can gate VERIFIER_ROLE in all other modules

How to Use:

Set this as a manager contract for VERIFIER_ROLE

Add OpenAI or Chainlink oracle runners with metadata like:

Name: OpenAI

Version: GPT-4-0613

Description: Used for SoGood tipping score evals

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts
/access/AccessControl.sol";

/**
 * @title AIOracleManager
 * @dev Register, activate, and manage
trusted AI verifier agents for Sotility
modules.
 */
contract AIOracleManager is
AccessControl {
    bytes32 public constant
ORACLE_ADMIN_ROLE =
keccak256("ORACLE_ADMIN_ROLE");

    struct Oracle {
        string name;
        string version;
        string description;
        bool active;
    }

    mapping(address => Oracle) public
oracles;
    address[] public oracleList;

    event OracleRegistered(address
indexed oracle, string name, string
version);
    event OracleStatusChanged(address
indexed oracle, bool active);

    constructor() {

_grantRole(DEFAULT_ADMIN_ROLE,
msg.sender);
        _grantRole(ORACLE_ADMIN_ROLE,
msg.sender);
    }

    function registerOracle(
        address oracle,
        string calldata name,
        string calldata version,
        string calldata description
    ) external
onlyRole(ORACLE_ADMIN_ROLE) {
        require(oracles[oracle].active ==
false, "Already registered");

        oracles[oracle] = Oracle({
            name: name,
            version: version,
            description: description,
            active: true
        });
        oracleList.push(oracle);
        emit OracleRegistered(oracle,
name, version);
    }

    function setOracleStatus(address
oracle, bool active) external
onlyRole(ORACLE_ADMIN_ROLE) {
        require(bytes(oracles[o
racle].name).length > 0, "Not
found");
        oracles[oracle].active = active;
        emit OracleStatusChanged(oracle,
active);
    }

    function isActiveOracle(address
oracle) public view returns (bool) {
        return oracles[oracle].active;
    }

    function getOracle(address oracle)
public view returns (Oracle memory) {
        return oracles[oracle];
    }

    function getAllOracles() external view
returns (address[] memory) {
        return oracleList;
    }
}
```

Notein
Scan the QR code to download Notein